FAST National University of Computer and Emerging Sciences, Lahore

Artificial Intelligence — Semester Project

Resumatch

AI-Powered Resume–Job Matching Platform

Deliverable 1: Problem Definition & Data Foundations

Track A — Application Development

Member	Roll No. / Section
Muhammad Sufyan	24L-2601 / BSDS-5B
[Member 2 name]	[Roll no. / Section]
[Member 3 name]	[Roll no. / Section]

October 2026

Contents

1 Introduction

Phase 1: Problem Definition

2 Problem Statement

3 Target Audience

4 AI Justification

5 Proposed System and Features

Phase 2: Data Acquisition & Preprocessing

6 Data Collection

7 Data Cleaning Implementation

8 Feature Strategy

9 Exploratory Data Analysis

10 Repository Structure

11 Status and Next Steps

12 Risks and Mitigations

13 Team Contribution Plan

References

1 Introduction

Resumatch is a web application that matches resumes and job postings in both directions. Job seekers upload a CV and receive ranked job recommendations, an ATS-style compatibility score, a skill-gap report, and rewritten resume bullets. Recruiters upload a job description together with a batch of CVs and receive a ranked, explained shortlist. The matching, ranking, and skill-extraction components are models we train or fine-tune ourselves; external LLM APIs are used only in supporting roles (pre-labelling, synthetic data generation, and natural-language explanations).

This document covers Phase 1 (Problem Definition) and Phase 2 (Data Acquisition & Preprocessing). Unlike a plan, every number reported in Phase 2 was produced by the scripts in the project repository (`data_scripts/`) and is regenerated automatically into this report from `report/stats/*.json`.

Phase 1: Problem Definition

2 Problem Statement

Hiring is a two-sided matching problem that both sides handle poorly.

For candidates. Many applicants, especially fresh graduates, write one generic CV and send it everywhere, and rarely learn why they were not shortlisted. Applications are first screened by Applicant Tracking Systems (ATS) and by recruiters who spend very little time per CV, so a strong candidate can be filtered out because the resume does not surface the skills a posting asks for, or describes them in different words ("built REST services" versus "API development").

For recruiters. A single posting can attract far more applications than can be read carefully. Keyword filters are the usual remedy, but they reward keyword stuffing and miss capable candidates who phrase their experience differently.

The gap. Existing tools are mostly either (a) simple keyword counters or (b) paid, closed products that give a score without explaining it. There is little transparent, evaluable tooling that (i) ranks in both directions, (ii) explains the specific missing skills, and (iii) is audited for bias against attributes such as name, gender, or university. Our own data (Section 9) confirms the vocabulary problem: 976 of the 2,278 distinct skills requested in postings never appear in any resume, and 22 of the 100 most requested skills appear in fewer than 1% of resumes.

3 Target Audience

- **Primary:** students and fresh graduates applying for internships and entry-level roles in Pakistan, who need actionable feedback on how well their CV fits a role.
- **Secondary:** early-career professionals switching roles or domains who need to see their skill gaps.
- **Secondary:** recruiters and HR staff at small companies and startups without an enterprise ATS, who need a fast, explainable shortlist.
- **Academic:** the bias-audit component addresses an open research concern in automated hiring tools.

4 AI Justification

A rule-based system cannot solve this problem well, for four reasons:

1. **Semantic variation.** The same skill appears in many surface forms (“JS”, “JavaScript”, “Node.js”; “ML”, “machine learning”). ESCO alone lists 99,192 alternative labels for 13,896 skills; a keyword system needs an exhaustive and constantly outdated synonym list. Sentence embeddings place semantically similar text close together without hand-written rules.
2. **Context and level.** “Used Python once in a course” and “led a Python team for three years” share a keyword but signal very different fit. A learned model can use context; a keyword counter cannot.
3. **Skill extraction from unstructured text.** CVs and postings have no fixed format. Named-entity recognition (NER) models learn to tag skills from examples instead of relying on brittle regular expressions. Our dictionary baseline (Section 8) needed a hand-built list of ambiguous words (“it”, “current”, “balance”, “chef”) to avoid false matches, which illustrates exactly this brittleness.
4. **Ranking is a learned preference.** A “good match” combines many weak signals (skills, seniority, domain, experience). A trained ranker can learn their weights from labelled pairs; hand-set weights cannot be validated.

Evidence from our data. For every resume we mined *hard negatives*: postings from a different occupation that are lexically close to the resume. They are as similar to the resume as true same-occupation *positives* (mean TF-IDF cosine 0.13 vs. 0.12). Because hard negatives are mined with TF-IDF, that baseline is disadvantaged by construction (AUC 0.40). The independent, off-the-shelf MiniLM sentence encoder does only slightly better, at AUC 0.53, barely above chance. Telling the right role from a similar-sounding wrong one therefore needs encoders fine-tuned on our pairs, which is the Phase 3 plan. Machine learning also allows measurement: precision@k, NDCG, and bias tests can be computed for a trained model and compared against keyword baselines.

5 Proposed System and Features

#	Feature	Description	Core model
1	Match score	Score and explanation for one CV versus one job	Cross-encoder
2	Skill-gap report	Skills present, missing, and a suggested learning order	Skill NER + ESCO
3	Job ranking	Top-N jobs from the dataset for an uploaded CV	Fine-tuned bi-encoder
4	Candidate ranking	Ranked shortlist of CVs for one job (recruiter mode)	Bi-encoder + reranker
5	ATS score	Section completeness, keyword coverage, formatting	Feature-based model
6	Bullet rewriter	Rewrites weak CV lines for a target job	LLM (API)
7	Skill normalization	Maps variants to a standard skill taxonomy	NER + ESCO mapping
8	Evaluation & bias dashboard	Metrics, baselines, counterfactual bias tests	—

Table 1: Planned features and the model behind each.

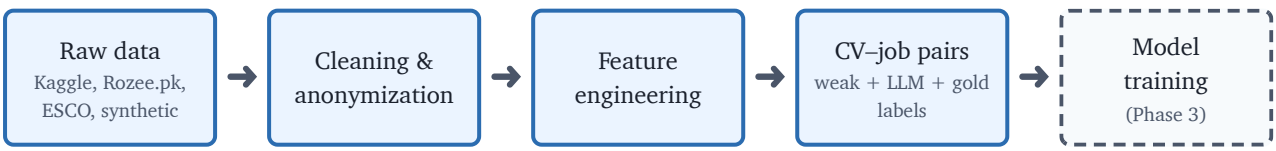


Figure 1: Data pipeline. Solid boxes are implemented in this deliverable; model training is Phase 3.

Role of external APIs. Pretrained LLM APIs are used to (i) pre-label CV–job pairs and generate synthetic CVs for training data, (ii) act as a zero-shot baseline in the evaluation, and (iii) produce user-

facing explanations and rewrites. The retrieval (bi-encoder [6]), reranking (cross-encoder [7]) and skill-extraction models are our own fine-tuned models. Every API call is logged.

Success criteria. Job and candidate ranking: NDCG@10 and precision@5 on the human gold set, compared with BM25/TF-IDF and zero-shot LLM baselines. Skill extraction: span-level F1 against the ESCO dictionary baseline. Bias: change in score under counterfactual swaps of name, gender, and university should be statistically indistinguishable from zero.

Phase 2: Data Acquisition & Preprocessing

6 Data Collection

Source	Content	Method	Raw	Clean
Kaggle *Resume Dataset* [1]	Resume text, 24 job categories	Download	2,484	2,473
Kaggle *Updated Resume Dataset* [2]	Resume text, 25 tech-heavy categories	Download	962	159
Kaggle *LinkedIn Job Postings* [3]	Title, description, location, level, salary	Download	33,246	27,738
Rozee.pk postings	Pakistan-based postings incl. skills tags, career level, city	Python scraping	352	348
ESCO v1.1.2 skills [4]	13,896 skills, 99,192 alternative labels	Download (CSV)	13,896	—
Resume–JD fit pairs [5]	Human-labelled *No / Potential / Good Fit* pairs	Download	6,241 + 1,759	held out
Synthetic CVs	LLM-generated CVs for under-represented roles	API calls (logged)		script ready
LLM-labelled pairs	*poor / partial / good* labels for built pairs	API calls (logged)		script ready

Table 2: Data sources with record counts before and after cleaning (Section 7).

Evidence of collection. Each source has a script in `data_scripts/`: `download_kaggle.py` (official Kaggle API when credentials exist, otherwise byte-identical public mirrors on the Hugging Face Hub), `download_esco.py`, `scrape_jobs.py`, `generate_synthetic_cvs.py`, and `label_pairs.py`. A download manifest records file names, sizes, row counts and origin. Every LLM call goes through `llm_client.py`, which appends the prompt, model, response, token usage and timestamp to `logs/api_calls.jsonl` and caches responses so no call is paid for twice. The client works with any OpenAI-compatible endpoint (OpenAI, Groq, Gemini).

Scraping policy. The scraper downloads and obeys `robots.txt` for every URL, waits 2–3 s between requests (adaptive), sends a descriptive user agent, doubles its delay on HTTP 403/429, and stores only public posting text (title, company, city, description, skills tags, career level, experience). It reads structured data from the schema.org `JobPosting` block on each page rather than brittle HTML selectors. Postings were collected for 19 of 33 planned search queries (software, data, design, finance, banking, marketing, HR and sales). When the site began rate-limiting (87 of 556 requests answered HTTP 403), the scraper backed off and, after five consecutive blocks, stopped as the policy requires. The remaining queries (engineering, healthcare, education, internships) are prioritized automatically

on the next run, so the current Pakistani sample is tech-heavy. All requests are logged to `logs/scrape_rozee.log`. Data is used for academic purposes only; if a site disallows scraping we rely on Kaggle data for that segment.

Gold set. `sample_gold_set.py` draws 200 pairs *only from the test split* (50% positives, 30% hard negatives, 20% random negatives; pair type hidden from annotators). Two members label them independently using written guidelines, and the script reports Cohen's κ (plain and linearly weighted) and flags disagreements for adjudication. This set is never used for training.

7 Data Cleaning Implementation

All cleaning is implemented in Python (pandas, regex, scikit-learn) in `data_scripts/clean_resumes.py` and `clean_jobs.py`, with shared utilities in `common.py`. Table 3 lists each step and its measured effect.

Issue	Handling	Resumes	Postings
Loaded	All sources merged into one schema	3,446	33,598
Missing values	Drop records with empty text or title. Fill missing categorical fields (location, seniority, work type) with `unknown`. Salary is never a model input; it is normalized to yearly values for analysis and left as NaN when missing (39.8% available).	−1	−1
Encoding noise	Repair mis-decoded UTF-8 (“NaÃ¨ve” → “Naïve”), NFKC normalization	38 fixed	—
Noise	Strip HTML tags and entities, URLs, EEO/legal boilerplate, non-printable characters	all	all
Non-English	Stop-word ratio test (English vs. Spanish/French/German/Portuguese)	−0	−23
Exact duplicates	SHA-1 hash of normalized text	−798	−3,236
Near-duplicates	TF-IDF cosine similarity >0.95 (reposted postings, copied CV templates)	−8	−2,087
Outliers	Flag below 1st / above 99th percentile of word count (IQR fences as cross-check: 222 / 532 flagged). Drop clear junk (resumes <50, postings <30 words); truncate long texts at the 99th percentile	−7; 27 capped	−165; 283 capped
Personal data	Replace emails, phone numbers, URLs and street addresses with `[EMAIL]`, `[PHONE]`, `[URL]`, `[ADDRESS]` before any storage or API call (optional spaCy NER for names)	117 masked	4,818 masked
Final		**2,632**	**28,086**

Table 3: Cleaning steps and their measured effect (records removed or modified).

Notable findings: the *Updated Resume Dataset* is mostly duplicates (796 of its 962 rows), and the Kaggle resumes still contained real-looking phone numbers and addresses (299 tokens masked), so anonymization is necessary rather than precautionary. Removed near-duplicate pairs are saved for inspection, not silently discarded.

8 Feature Strategy

Implemented in `data_scripts/build_features.py` and `skills.py`. All fitted transformers (scaler, encoder, TF-IDF vocabulary) are fitted on the *training split only*.

- **Text normalization:** lowercasing, punctuation cleanup and light rule-based lemmatization for the TF-IDF baseline only; transformer inputs keep original casing and punctuation.
- **Text representations:** TF-IDF over unigrams and bigrams (50,000 features, sublinear TF) as the baseline, and 384-dimensional L2-normalized sentence embeddings from `all-MiniLM-L6-v2` [6], to be fine-tuned in Phase 3.
- **Skill extraction and normalization:** a gazetteer of ESCO preferred and alternative labels plus curated tech synonyms maps variants ("JS", "ReactJS", "k8s") to one canonical skill. Ambiguous single words are excluded. This yields on average 7.95 skills per resume and 3.91 per posting, and is the baseline the Phase 3 NER model must beat.
- **Encoding:** resume categories and posting categories (posting titles mapped to the same 24 categories by ordered rules, covering 60.9% of postings) are label-encoded in a shared label space; skills are multi-hot encoded (`MultiLabelBinarizer`) over a vocabulary of 1,169 ESCO skills seen in at least five training documents.
- **Numeric features:** text length, number of skills, and years of experience. Years are extracted by regex from explicit mentions ("5+ years of experience") or, for resumes, from the span of dated job entries, with "Current" anchored to the collection year of the source (2019) rather than today (coverage: 95.8% of resumes, 46.9% of postings). Heavy-tailed counts are log-transformed, missing years get a train-median value plus a missing indicator, and all are standardized with `StandardScaler`.
- **Pair construction:** for each resume, positives are the 3 most similar postings of the same category; hard negatives are the 2 most similar postings of a *different* category; one random different-category posting is added. This gives 15,755 pairs (7,859 positive, 5,264 hard negative, 2,632 random negative), balanced 50/50.
- **Splits:** 70/15/15 train/validation/test split *by resume and by posting* (not by pair), stratified by category, and pairs are only formed within a split, so no resume or posting leaks across splits. Resumes: 1,842 / 395 / 395. Postings: 19,660 / 4,213 / 4,213. Pairs: 11,052 / 2,351 / 2,352.

9 Exploratory Data Analysis

All charts are produced by `data_scripts/eda.py` on the cleaned data.

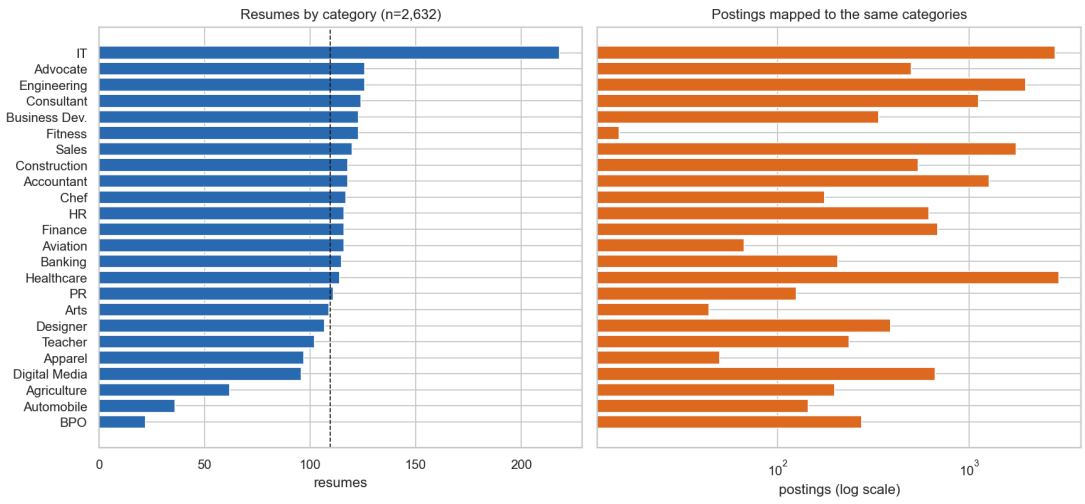


Figure 2: Class distribution of resume categories (left) and postings mapped to the same categories (right, log scale). Most resume classes hold 100–125 samples, but the imbalance ratio is 9.91 (IT: 218 vs. BPO: 22), so we will use class weights, stratified splits and synthetic CVs for the rare roles.

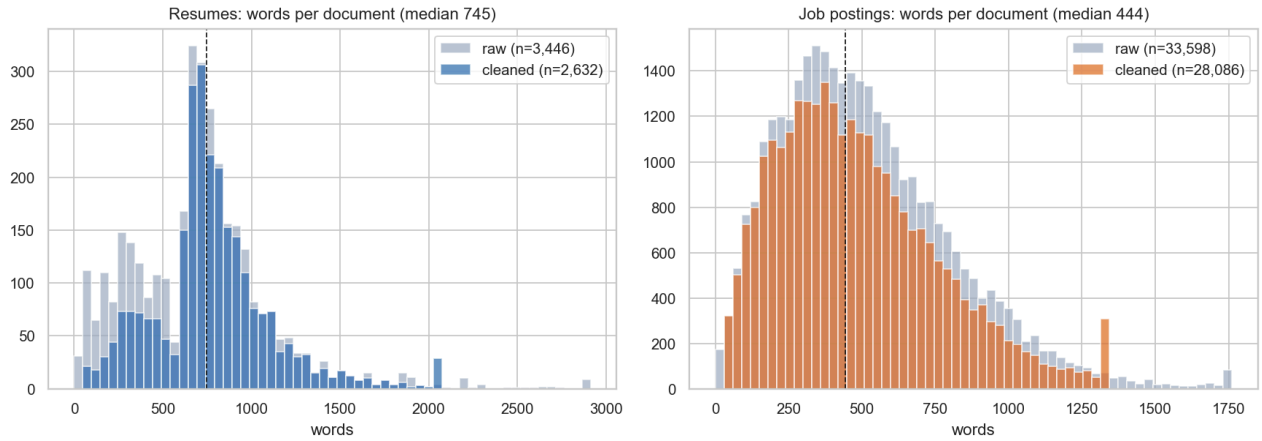


Figure 3: Text length before (grey) and after cleaning. The median resume has 745 words and the median posting 444. Most short raw resumes disappear because they are duplicates; the spike at the right edge of the posting histogram is the 99th-percentile cap.

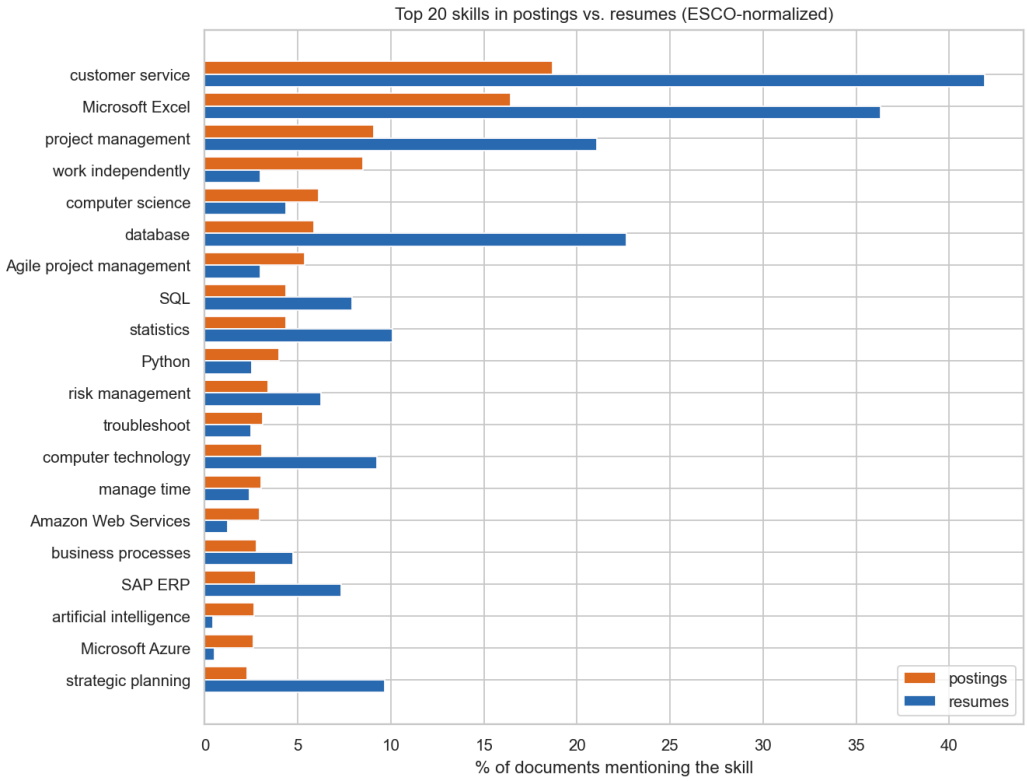


Figure 4: Top 20 skills in postings versus resumes. Resumes over-emphasize generic skills (customer service, Excel), while postings ask for cloud, AI and Agile skills that resumes rarely mention. Of the 2,278 distinct skills found in postings, 976 never occur in any of the 2,632 resumes (Jaccard overlap of the two vocabularies: 0.53), and 22 of the 100 most requested skills appear in fewer than 1% of resumes: the vocabulary mismatch the skill-gap report targets.

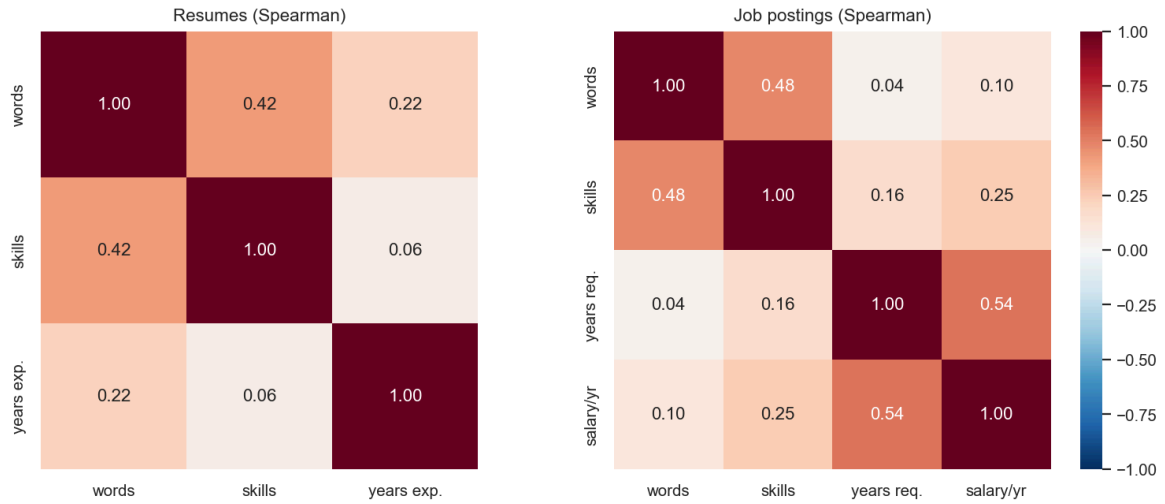


Figure 5: Spearman correlations of numeric features. Longer documents list more skills (ρ=0.42 for resumes, 0.48 for postings), and required years of experience is the strongest correlate of advertised salary (ρ=0.54). No pair is collinear enough to drop a feature.

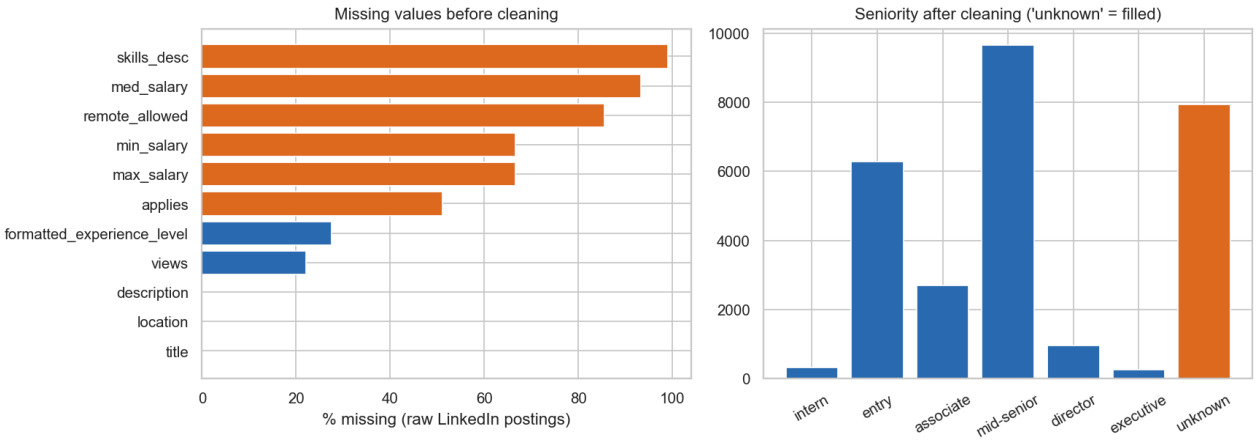


Figure 6: Missing values in the raw LinkedIn data (left) and seniority after cleaning (right). Salary is missing for most postings and is therefore not a model input; 27.3% of seniority labels are missing and are filled with `unknown` instead of being guessed.

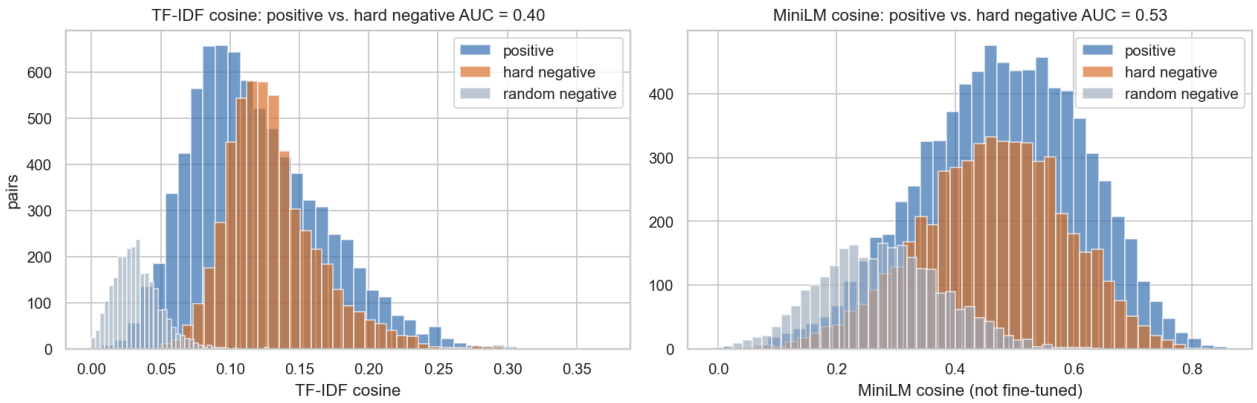


Figure 7: Similarity of constructed pairs under TF-IDF (left) and the pretrained, not yet fine-tuned MiniLM encoder (right). Random negatives are easy to separate, but hard negatives overlap almost completely with positives (MiniLM mean cosine 0.46 vs. 0.48; AUC 0.53). These pairs are exactly the training signal the Phase 3 rankers need.

Further observations. (1) The Kaggle resumes are written by experienced professionals (median well above ten years of experience), while our primary users are fresh graduates; synthetic entry-level

CVs and the Pakistani postings address this domain gap. (2) The LinkedIn postings are US-based. The Rozee.pk data adds local titles, cities and skill tags; its most requested skills are artificial intelligence, Python, computer science, Git, CI/CD. (3) 8 of the 348 Rozee.pk postings state an explicit gender preference (7 male, 1 female). We keep this field for the Phase 4 bias analysis and never use it as a model input.

10 Repository Structure

```
resumatch/
|-- README.md               # setup, pipeline, data sources
|-- Makefile                # make data scrape clean features eda gold report
|-- requirements.txt
|-- .env.example            # LLM / Kaggle keys (real .env is git-ignored)
|-- .gitignore              # excludes raw data, models, API keys, API cache
|-- data_scripts/
|   |-- common.py           # paths, noise removal, PII masking, dedup helpers
|   |-- download_kaggle.py  # Kaggle API or HF mirrors + manifest
|   |-- download_esco.py
|   |-- scrape_jobs.py      # Rozee.pk, robots.txt-aware, rate-limited
|   |-- llm_client.py       # OpenAI-compatible client, JSONL logging + cache
|   |-- generate_synthetic_cvs.py   # API, logged
|   |-- label_pairs.py              # API, logged
|   |-- clean_resumes.py
|   |-- clean_jobs.py
|   |-- skills.py           # ESCO gazetteer: extraction + normalization
|   |-- build_features.py   # features, splits, pairs, embeddings
|   |-- sample_gold_set.py  # gold set sampling + Cohen's kappa
|   `-- eda.py
|-- data/                   # raw/ interim/ processed/ gold/ (git-ignored)
|-- logs/                   # api_calls.jsonl, scrape log
|-- notebooks/
|-- report/                 # LaTeX source, figures, stats, PDF build
`-- app/                    # Phase 5 (Streamlit/Gradio)
```

11 Status and Next Steps

Item	Status
Kaggle resumes and postings, ESCO taxonomy downloaded	Done
Rozee.pk scraper with robots.txt, rate limiting and logging	Done (352 postings)
Cleaning, anonymization, deduplication, outlier handling	Done
Skill gazetteer, TF-IDF, multi-hot, numeric features, MiniLM embeddings	Done
Leakage-free splits and 15,755 weakly labelled pairs	Done
EDA figures	Done
Synthetic CVs and LLM pair labels (scripts and logging ready)	Pending API key
Gold set: 200 pairs sampled; double annotation and κ	Pending manual labelling

Table 4: Deliverable 1 status.

12 Risks and Mitigations

Risk	Mitigation
Scraping blocked or disallowed	robots.txt checked per URL; fall back to Kaggle postings; keep the scraper limited to permitted pages.
Noisy weak / LLM labels	Hand-verify a sample, report agreement, and keep the gold test set human-labelled.
Small or imbalanced resume data	Synthetic CVs for rare roles, class weighting, stratified splits.
Domain gap (US, senior CVs vs. Pakistani graduates)	Rozee.pk postings, synthetic entry-level CVs, separate evaluation on local data.
Bias in data (gender, name, institution)	PII anonymized; posting gender preferences kept for auditing only; counterfactual swap tests in Phase 4.
API cost or rate limits	All responses cached; a small, cheap model for bulk labelling; exponential back-off.
Limited compute	Small models (MiniLM-class) trainable on a free Colab GPU.

13 Team Contribution Plan

Member	Responsibility in Deliverable 1
Muhammad Sufyan	Repository setup, scraping scripts, data cleaning pipeline, report
[Member 2]	Kaggle data cleaning review, EDA, gold-set annotation (annotator 1)
[Member 3]	Feature engineering, synthetic CV generation, gold-set annotation (annotator 2)

References

1. snehaanbhawal (Kaggle user). *Resume Dataset*. Kaggle. https://www.kaggle.com/datasets/snehaanbhawal/resume-dataset
2. gauravduttakiit (Kaggle user). *Resume Dataset (UpdatedResumeDataSet.csv)*. Kaggle. https://www.kaggle.com/datasets/gauravduttakiit/resume-dataset
3. arshkon (Kaggle user). *LinkedIn Job Postings (2023–2024)*. Kaggle. https://www.kaggle.com/datasets/arshkon/linkedin-job-postings
4. European Commission. *ESCO: European Skills, Competences, Qualifications and Occupations*, v1.1.2. https://esco.ec.europa.eu
5. *resume-job-description-fit*. Hugging Face Datasets. https://huggingface.co/datasets/cnamuangtoun/resume-job-description-fit
6. N. Reimers and I. Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP*, 2019.
7. R. Nogueira and K. Cho. Passage Re-ranking with BERT. arXiv:1901.04085, 2019.